AIM :

To implement the Map Coloring Problem using Constraint Satisfaction Problem (CSP) in Python.

ALGORITHM :

1. Start the program.
2. Define the map regions as variables.
3. Assign three colors to each region as domains.
4. Define neighboring regions as constraints.
5. Select an unassigned region.
6. Check whether the selected color satisfies the constraints.
7. If valid, assign the color and continue recursively.
8. If invalid, backtrack and try another color.

9. Repeat until all regions are colored.

10. Print the solution.

11. Stop.

PROGRAM :

File Edit Format Run Options Window Help

```
# Map Coloring using CSP

variables = ['A', 'B', 'C', 'D']

domains = {
    'A': ['Red', 'Green', 'Blue'],
    'B': ['Red', 'Green', 'Blue'],
    'C': ['Red', 'Green', 'Blue'],
    'D': ['Red', 'Green', 'Blue']
}

constraints = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

assignment = {}


def is_valid(variable, color):
    for neighbor in constraints[variable]:
        if neighbor in assignment:
            if assignment[neighbor] == color:
                return False
    return True


def backtrack():
    if len(assignment) == len(variables):
        return True

    for variable in variables:
        if variable not in assignment:
            for color in domains[variable]:
                if is_valid(variable, color):
                    assignment[variable] = color

                    if backtrack():
                        return True

                    del assignment[variable]

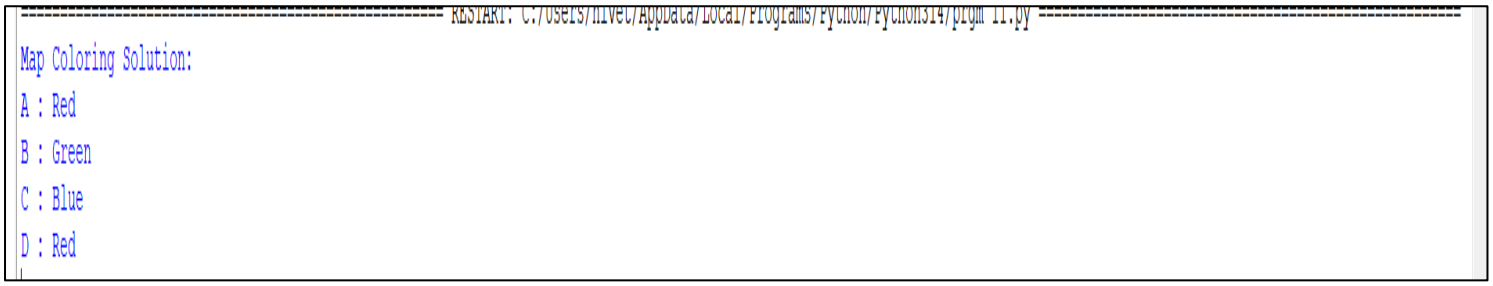
            return False
```

```
            return False

    return True


if backtrack():
    print("Map Coloring Solution:")
    for variable, color in assignment.items():
        print(variable, ":", color)
else:
    print("No solution exists")
```

OUTPUT :



RESULT :

Thus, the Map Coloring Problem was successfully implemented using the Constraint Satisfaction Problem (CSP) with the Backtracking Algorithm in Python.